

TightLLM: Maximizing Throughput for LLM Inference via Adaptive Offloading Policy

Yitao Hu , Xiulong Liu , Senior Member, IEEE, Guotao Yang , Linxuan Li , Kai Zeng ,
Zhixin Zhao , Graduate Student Member, IEEE, Sheng Chen , Member, IEEE,
Laiping Zhao , Member, IEEE, Wenxin Li , and Keqiu Li , Fellow, IEEE

Abstract—Large language models (LLMs) have demonstrated remarkable performance across a wide range of tasks, largely due to their substantial model size. However, this also results in significant GPU memory demands during inference. To address these challenges on hardware with limited GPU memory, existing approaches employ offloading techniques that offload unused tensors to CPU memory, thereby reducing GPU memory usage. Since offloading involves data transfer between GPU and CPU, it introduces transfer overhead. To mitigate this, prior works typically overlap data transfer with GPU computation using a fixed pipelining strategy applied uniformly across all inference iterations, referred to as *static* offloading. However, static offloading policies fail to maximize inference throughput because they cannot adapt to the dynamically changing transfer overhead during the inference process, leading to increasing GPU idleness and reduced inference throughput. We propose that offloading policies should be *adaptive* to the varying transfer overhead across inference iterations to maximize inference throughput. To this end, we design and implement an adaptive offloading-based inference system called TightLLM with two key innovations. First, its key-value (KV) distributor employs a *trade-compute-for-transfer* strategy to address growing transfer overhead by dynamically recomputing portions of the KV cache, effectively overlapping data transfer with computation and minimizing GPU idleness. Second, TightLLM’s weight loader slices model weights and distributes the loading process *across multiple batches*, amortizing the excessive weight loading overhead and significantly improving throughput. Evaluation across various combinations of GPU hardware and LLM models shows that TightLLM achieves 1.3 to 23 times higher throughput during the decoding phase and 1.2 to 22 times higher throughput in the prefill phase compared to state-of-the-art offloading systems. Due to the higher throughput in prefill and decoding phases, TightLLM can reduce the completion time for large-scale tasks, which involve processing and generating a substantial number of tokens, by 59.6% to 94.9%.

Index Terms—Large language models (LLMs) inference, offloading, throughput-oriented.

Received 31 December 2024; revised 23 February 2025; accepted 15 March 2025. Date of publication 8 April 2025; date of current version 11 June 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4501000 and in part by the National Natural Science Foundation of China under Grant 62202328. Recommended for acceptance by X. Fu. (Corresponding author: Xiulong Liu.)

The authors are with Tianjin Key Laboratory of Advanced Networking, Department of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: xiulong_liu@tju.edu.cn).

Digital Object Identifier 10.1109/TC.2025.3558009

I. INTRODUCTION

RECENTLY, large language models (LLMs) have demonstrated exceptional performance across a variety of tasks, including text generation, question answering and language translation [1], [2], [3]. While these models are widely used for interactive applications, they are increasingly employed in non-interactive tasks such as literature translation, benchmarking and form processing [4], [5], [6]. For these non-interactive tasks, service providers prioritize throughput over latency, as these tasks are not time-sensitive. Consequently, LLM service providers like OpenAI [7] and Anthropic [8] have introduced specialized throughput-oriented APIs, whose primary objective is to maximize throughput within relaxed time constraints (e.g., 24 hours).

At the same time, these non-interactive tasks often demand local deployment for privacy and security concerns [9], [10], [11], which requires serving LLM models on individual GPU. However, individual GPU often lacks sufficient GPU memory for LLM inference. For instance, modern GPU, such as NVIDIA’s 3090 or A800, offers between 24 to 80 GB of GPU memory, which often falls short of the over 100 GB needed for deploying the latest LLM models such as GPT-4 [12] or Llama 3 [13]. This gap between the available GPU memory on individual hardware and the required GPU memory for LLM inference presents a significant barrier to widespread adoption. Therefore, in this paper, we focus on high-throughput LLM inference on individual GPUs, where extensive GPU memory is typically unavailable.

To address the gap between available GPU memory and the required GPU memory for LLM inference, researchers have been exploring multiple strategies. The first approach applies various model parallelism techniques [14], [15], [16], [17] to distribute the model across multiple GPUs. However, this method necessitates a multi-GPU setup, which is typically inaccessible to individual users. The second approach involves model compression or quantization techniques [18], [19], [20], [21], [22], which reduce GPU memory demands by decreasing model size or precision. While effective in lowering GPU memory requirements, the second method often incurs a trade-off in model accuracy.

A recently popular approach is to offload model weights and/or key-value (KV) cache from GPU memory to lower-tier

storage, such as CPU memory¹ [23], [24], [25], [26], [27], [28]. Specifically, when computing a transformer block in the LLM model, the offloading system keeps only the necessary portion (e.g., one transformer block) of model weight and KV cache in GPU memory, while leaving the remainder to CPU memory. Additionally, the offloading system often employs pipelining between computation and data transfer: when computing the current transformer block, it simultaneously loads the required data for the next block. By continuously transferring data between GPU and CPU memory, this offloading approach manages to deploy LLM models on individual hardware with limited GPU memory. This technique is particularly suitable for individual users, as it avoids the need for multi-GPU setups and prevents accuracy degradation, while typically being optimized for higher throughput [25], [26].

Existing offloading policies fail to maximize throughput due to their reliance on a static design, where *the pipelining strategy for computation and data transfer is fixed across all inference iterations*. Consequently, they are unable to adapt to the growing gap between computation and data transfer, which results in inefficient GPU resource usage and a reduction in throughput. Specifically, these static offloading policies fail to make throughput-maximum offloading decisions due to two limitations: (1) Inability to adapt to KV cache growth: Since KV cache grows with each decoding iteration, the time required to load it from CPU to GPU also increases, while computation time remains relatively constant. Consequently, during pipelining, the gap between computation and transfer widens progressively. Current offloading systems overlook this expanding gap, leading to idle computation time and inefficient resource utilization. (2) Inability to hide transfer overhead of model weights: Loading model weight from CPU to GPU incurs substantial transfer overhead, often much greater than the associated computation time. As a result, existing systems, which aim to overlap this transfer time with computation of a single batch², are unable to fully hide the transfer overhead, resulting in reduced throughput.

In this paper, we argue that **the offloading system should be adaptive to different transfer overheads across inference iterations, so as to maximize inference throughput**. Based on that, we design and implement an offloading-based inference system called TightLLM, which makes proper offloading decisions to maximize throughput as follows.

First, TightLLM introduces a KV distributor to address the bottleneck caused by the growing transfer overhead of the KV cache. Existing offloading systems always load the *entire* KV cache from CPU to GPU memory, leading to expanding transfer overhead as decoding progresses. Instead, for each iteration, TightLLM's KV distributor dynamically determines a *partial* part of the KV cache to load from the CPU and recomputes the rest on the GPU directly. Specifically, during the decoding phase, the KV distributor dynamically selects the KV cache of partial tokens for recomputation, so as to balance computation

and transfer overhead based on changes in KV cache size across inference iterations. In the prefill phase, the KV distributor does not offload the KV cache that needs to be recomputed in the following decoding phase from GPU to CPU, so as to reduce the transfer overhead. By doing so, TightLLM will have less transfer overhead and more computation operation due to KV recomputation, which we call *trade-compute-for-transfer*. With a proper amount of recomputation, TightLLM is able to reduce transfer overhead, which avoids computation idleness, thereby leading to higher throughput.

Second, TightLLM introduces a weight loader to mitigate the bottleneck associated with the substantial transfer overhead of model weight. Existing offloading systems always transfer the model weight for the next transfer block during the computation of *a single batch* for the current block, which cannot hide the transfer overhead due to the excessive parameter size in the transformer block. Instead, since the model weight for each block needs to be transferred only once across multiple batches, TightLLM's weight loader slices the model weight of each block and transfers the sliced model weight *across multiple batches*, so as to amortize the excessive parameter size across them. By doing so, TightLLM manages to overlap the transfer overhead for the sliced model weight with computation, leading to higher throughput.

When maximizing inference throughput, TightLLM makes the following contributions:

- TightLLM identifies the importance of dynamic offloading policy on throughput maximization and provides a systematic analysis of how existing static offloading policies fail to achieve high throughput.
- TightLLM designs an adaptive offloading policy with its KV distributor and weight loader to address transfer bottlenecks related to KV cache and model weight, so as to maximize the throughput for LLM inference.
- Evaluation shows that TightLLM achieves throughput improvements by $1.3\times$ to $23\times$ in the decoding phase and $1.2\times$ to $22\times$ in the prefill phase compared to the state-of-the-art. Additionally, it significantly reduces completion latency for large-scale tasks by 59.6% to 94.9%. The ablation study further quantifies the importance of TightLLM's design choices in maximizing throughput.

II. BACKGROUND

LLM Architecture and Inference. Recent LLMs [13], [30], [31], [32] often employ a decoder-only transformer architecture composed of multiple transformer blocks. Each transformer block consists of an attention layer followed by a feed-forward network (FFN). The attention layer assigns weights to input tokens, allowing the model to focus on relevant information and capture contextual relationships between tokens. Subsequently, FFN applies transformations to these weighted representations to learn complex patterns and features. The output of each FFN layer then serves as input to the next transformer block, progressively refining the representation. After processing through all blocks, LLM produces a probability vector representing the most likely next token.

¹To distinguish it from GPU memory, we refer to random access memory (RAM) as CPU memory.

²Batching [25], [29] is widely used to group multiple requests together to leverage GPU's parallel computing capability for higher throughput.

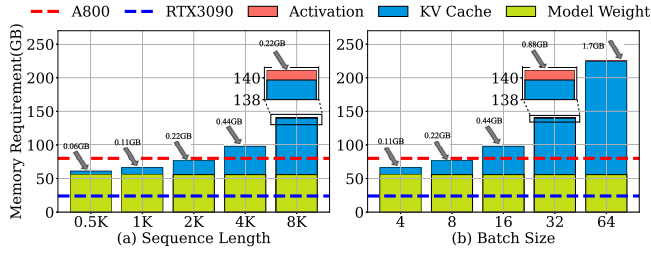


Fig. 1. Total memory requirement of OPT-30B under different sequence lengths and batch sizes. The batch size in (a) is 8, while the sequence length in (b) is 2K. Gray arrows point to the activation of memory usage.

LLM inference involves two main phases: prefill and decoding. In the *prefill* phase, LLM processes the input tokens and generates the first token of the response in a single iteration. In the subsequent *decoding* phase, LLM generates one token per iteration, using the previously generated tokens to produce the next one in an auto-regressive manner. During both phases, the inference system repeatedly uses the key (K) and value (V) matrices from preceding tokens to generate subsequent tokens. The inference system typically caches these matrices to avoid redundant computation, called KV cache.

Memory Footprint. LLM inference consumes GPU memory across three main categories: (1) *Model weights*. Following the model scaling laws [33], modern LLMs use an increasingly large number of parameters to improve performance. As a result, model weights occupy a significant portion of total memory usage during inference and remain constant for each specific LLM. (2) *KV cache*. As mentioned earlier, the KV cache reduces redundant computation by trading storage for computation. The GPU memory allocated to the KV cache is typically released only after the entire input has been finished [29]. (3) *Intermediate activation*. It is the variable passed between layers, such as from the attention layer to the FFN layer within a transformer block or between consecutive transformer blocks [25].

Among these three categories, the GPU memory required for the KV cache grows linearly with both sequence lengths and batch sizes, while model weights occupy a fixed portion of memory and intermediate activation consumes a relatively negligible amount. For instance, Fig. 1 illustrates the GPU memory consumption for each category under varying sequence lengths and batch sizes when running the OPT-30B model [34]. As the sequence length increases from 0.5K to 8K, memory usage rises from 61 GB to 140 GB. Likewise, increasing the batch size from 4 to 64 results in a rise in memory usage from 66 GB to 225 GB.

The immense GPU memory requirements for LLM inference often surpass the capacity of individual GPUs. For instance, the required GPU memory of OPT-30B for *any* sequence length or *any* batch size exceeds the 24 GB memory capacity of 3090 GPU (dotted blue line in Fig. 1). Meanwhile, the required GPU memory for sequence length over 2k or batch size over 8 exceeds the 80 GB memory capacity of A800 GPU (dotted red line in Fig. 1). Therefore, there is a huge gap between the available GPU memory on individual hardware and the required one for LLM inference.

LLM Offloading. Given the memory gap, existing systems address this issue using multiple approaches. As discussed in Section I, parallel techniques [14], [15], [16], [17] need a multi-GPU setup while model compression or quantization [18], [19], [20], [21] come at the cost of accuracy, making them less applicable. A recently popular approach is to offload the model weight and KV cache³ from GPU memory to CPU memory, so as to deploy LLM models on individual GPU with limited GPU memory.

Specifically, existing offloading-based inference systems load only the necessary portions of model weights and KV cache required for the current inference step (*e.g.*, executing the first transformer block) into GPU memory while leaving the remainder to CPU memory. Once the current computation is complete, the system loads the model weights and KV cache needed for the next inference step (*e.g.*, executing the subsequent transformer block) into GPU memory, while offloading the newly generated KV cache to CPU memory. By continuously transferring the model weights and KV cache between GPU and CPU, LLM can complete the inference process while utilizing the limited GPU memory.

Throughput-oriented Offloading. While offloading increases inference latency due to the added overhead from data transfers between CPU and GPU, it is well-suited for non-interactive tasks. These tasks typically involve batch inference on large volumes of tokens (*e.g.*, translating an entire library of documents) and are less sensitive to the latency of individual requests. Despite the higher latency for individual requests caused by offloading, the system can still achieve a throughput of tens of tokens per second for such tasks. As a result, the primary goal of many offloading-based LLM inference systems is to maximize throughput [23], [24], [25], [26], [35].

Offloading introduces additional transfer overhead between the GPU and CPU, which negatively impacts throughput. To minimize this overhead and maximize throughput, existing systems leverage two techniques. (1) *Multi-batch Inference*: Offloading systems reduce the transfer overhead associated with loading weights by processing multiple batches with a single load of the transformer block. As shown in Fig. 2, a naive inference process computes all transformer blocks for one batch at a time, requiring a weight load for each batch and thus loading the weights multiple times. In contrast, the existing offloading system reduces the number of weight loads by processing multiple batches in one weight load, thereby enhancing inference throughput. (2) *Trans-comp Overlap*: Existing systems minimize the transfer overhead by overlapping data transfers with the computation. In Fig. 2, a naive transfer approach loads the required weights or KV cache on-demand before computation, which incurs excessive transfer overhead that blocks the computation of the current batch. Therefore, the existing offloading system leverages pipelining to preload the weight or KV cache during the previous batch's computation, so as to overlap transfer overhead and computation.

³Regardless of whether the offloading technique is used, the memory allocated and freed for intermediate activation on the GPU remains unchanged.

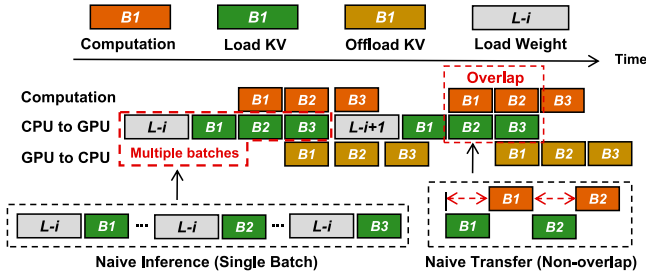


Fig. 2. To increase inference throughput, offloading systems optimize both the inference process and the transfer process through multi-batch inference and trans-comp overlap.

However, existing offloading policies adopt a *static* pipelining strategy to overlap data transfer and computation across inference iterations. We argue that such static policies cannot make optimal offloading decisions, leading to sub-optimal inference throughput, as we will discuss in detail in Section III.

III. MOTIVATION

In this section, we empirically examine the limitations of the offloading policies of existing systems (Section III-A) and provide an in-depth analysis of the underlying reason (Section III-B). Finally, we identify two key questions that an optimal throughput-oriented offloading policy should address (Section III-C).

A. Limitation of Static Offloading Policy

To highlight the limitations of current offloading systems for throughput-oriented LLM inference, we evaluate two state-of-the-art offloading systems Accelerate [24] and FlexGen [25] by serving OPT-175B model on an NVIDIA A800 GPU under sequence length (s_l) from 0.5K to 8K. Serving the OPT-175B model requires 343 GB ($s_l = 0.5K$) to 614 GB ($s_l = 8K$) of GPU memory with the batch size of 8, which far exceeds the memory capacity of 80 GB for A800 GPU. To validate that existing offloading systems cannot maximize throughput, we also include the result of TightLLM under the same setup. The primary evaluation metric is *throughput* in both the prefill and decoding phases.

We calculate the average normalized⁴ throughput over the given workload under various sequence lengths for existing offloading systems in Fig. 3 for comparison. From the results, it is notable that the throughput of FlexGen and Accelerate is 35.9%-95.4% and 60.4%-95.6% lower than that of TightLLM in the prefill and decoding phases, respectively. Besides, when the sequence length is 8K, Accelerate suffers from an out-of-memory issue, since it only offloads weight to the CPU while keeping all KV cache in GPU memory. All these results support our argument that existing offloading systems achieve sub-optimal throughput.

⁴We normalized the throughput for existing offloading systems over TightLLM's for a better presentation of their limitations.

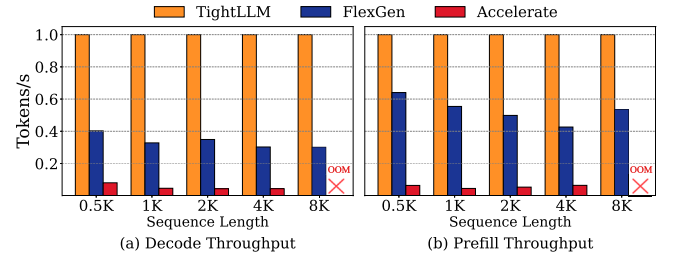


Fig. 3. The normalized token generation throughput of existing offloading systems compared to TightLLM across varying sequence lengths using a Tesla A800 GPU with OPT-175B.

B. Understand Why Static Offloading Policy Fails

We argue that the static computation and transfer strategy in existing offloading systems fails to effectively hide transfer overhead with computation, leading to sub-optimal inference throughput. This limitation arises from the fact that transfer overhead for model weights and KV cache changes dynamically across inference iterations, yet static offloading policies do not account for this variability. As a result, when transfer time exceeds computation time, the existing systems experience prolonged idle computation periods while waiting for data transfers to complete, ultimately reducing throughput. In the following, we analyze the underlying causes of the transfer bottleneck in both the decoding and prefill phases, shedding light on the factors that contribute to these performance limitations.

In the decoding phase: The transfer bottleneck arises from the weight loading and the KV cache loading. When loading weights or the KV cache, we transfer them from CPU to GPU, and when offloading the newly generated KV cache, we transfer it from GPU to CPU. As shown in Fig. 4, before the computation of the M_{th} iteration, the weight of the first transformer block and the KV cache of the first batch must be loaded from the CPU to the GPU. Subsequent batches only require loading the KV cache until the inference is transited to the next transformer block.

1) *For the first batch in block:* Its transfer overhead comes from the loading of both model weight and the KV cache. However, the loading time of a transformer weight is typically much longer than the time for its computation, which mainly consists of the attention and FFN layers. As a result, these computations cannot fully overlap with the transfer overhead, leading to a transfer bottleneck. For example, in Fig. 4, when loading the second transformer block during the iteration M , the weight loading time significantly exceeds the computation time of the last batch in the first transformer block, creating a transfer bottleneck and resulting in low throughput.

2) *For the non-first batch in block:* Its transfer overhead comes from only the loading KV cache and increases linearly with the number of generated tokens during the decoding phase. In the early iterations of the inference, the KV cache loading overhead is less than the computation overhead and can be entirely hidden. However, as inference continues, the KV cache loading overhead gradually surpasses the computation overhead, creating a transfer bottleneck. For example, in Fig. 4,

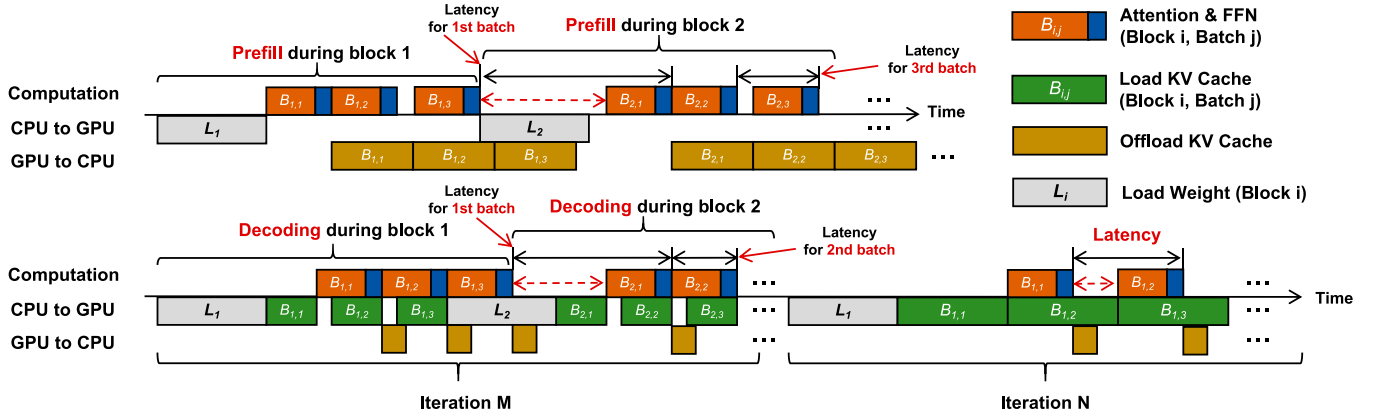


Fig. 4. Inference of existing systems with different iterations, blocks, and batches in the decoding phase. During the inference process, there are three levels of abstraction: iteration, block, and batch. Each token generation, referred to as an iteration, involves traversing each transformer block of the model. Furthermore, each transformer block processes multiple batches, as illustrated in Fig. 2.

the overhead of loading the KV cache grows rapidly from the M_{th} iteration to the N_{th} iteration, becoming a bottleneck of the inference and thus decreasing the throughput.

In the prefill phase: Similarly, the transfer overhead arises from both the model weights and the KV cache as shown in Fig. 4. However, unlike the decoding phase, the prefill phase does not require loading the KV cache from CPU to GPU; instead, it offloads the newly generated KV from GPU to CPU. Prefill phase generates a substantial number of KV caches that must be offloaded to CPU memory, resulting in high transfer overhead. In contrast, the decoding phase generates the KV cache one token at a time, leading to negligible overhead.

Additionally, since offloading the KV cache and loading the weights can occur simultaneously, the transfer bottleneck for the first batch in the block is variable and depends on the input length. When the input is long, the KV cache offload becomes the primary transfer bottleneck; conversely, when the input is short, loading the weights predominates as the main bottleneck.

C. Implications

The above analysis highlights that applying a *static* offloading policy cannot yield high throughput. To address the limitation of existing offloading systems and ensure high throughput for LLM inference, TightLLM should adopt a *adaptive* offloading policy, addressing two key questions: (1) How to adapt to the increasing KV cache loading time? Addressing this question requires narrowing the growing gap between computation time and KV cache loading time. This is particularly difficult because the KV cache expands during decoding, resulting in progressively longer loading time. (2) How to reduce the overhead of large weight loading time? Addressing this question involves concealing the weight loading time by overlapping it with computation. However, this is challenging due to the significant weight loading overhead, which far exceeds the time required for processing a single batch computation.

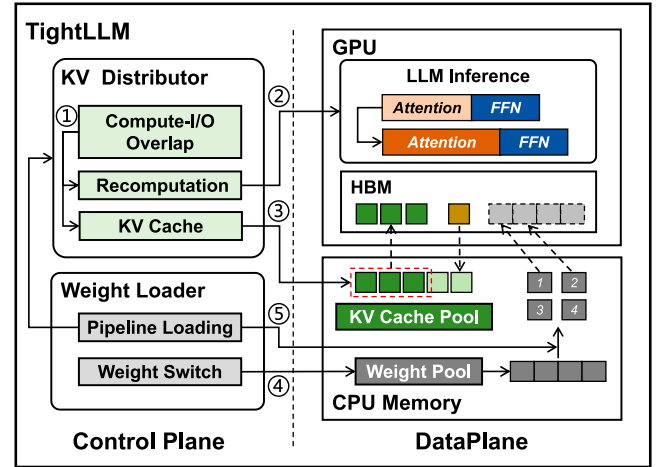


Fig. 5. TightLLM overview.

IV. SYSTEM DESIGN

We start with an overview of TightLLM (Section IV-A), followed by its two key designs, KV distributor (Section IV-B) and weight loader (Section IV-C), both of which are critical to guarantee high throughput. Finally, we put it all together to illustrate the lifecycle of how TightLLM maximizes throughput (Section IV-D).

A. System Overview

We design an offloading-based LLM inference system, TightLLM, which guarantees high throughput via *adaptive management of KV cache offloading and pipelined weight loading within each transformer block*. Fig. 5 illustrates the overall design of TightLLM, which includes two main components: (1) the KV distributor, which adaptively manages KV cache offloading to handle increased cache loading overhead as sequence length grows during the decoding phase; and (2) the weight loader, which amortizes substantial, infrequent weight-loading overhead across batches at the transformer block level, effectively concealing this overhead.

TightLLM manages offloading LLM inference, including loading and offloading model weight and KV cache between GPU HBM (which is an ultra-high data transfer rate DRAM technology designed to meet the huge data processing needs of GPUs) and CPU memory, as well as executing GPU computations. A complete copy of the model weight and the KV cache is stored in CPU memory, organized into a weight pool and a KV cache pool. TightLLM first invokes the KV distributor to optimize the KV computation ratio, maximizing the overlap between computation and data loading or offloading ①. After that, TightLLM controls the amount of GPU computations ② while dynamically loading or offloading the KV cache between GPU HBM and CPU memory ③. When weight switching is required (*i.e.*, across consecutive transformer blocks), TightLLM uses the weight loader to manage this process, minimizing loading overhead through optimized computation overlap. The weight loader divides each transformer block as needed from the CPU memory weight pool ④ and pipelines it to GPU memory ⑤ effectively concealing loading times with computation.

Since the KV cache must be loaded for each batch inference, while weights are only loaded during weight switching, we first introduce the KV distributor (Section IV-B) then the weight loader (Section IV-C), and finally put it all together (Section IV-D).

B. KV Distributor

Insight and Approach. To address the transfer bottleneck caused by excessive KV cache, TightLLM converts transfer overhead into compute overhead through KV cache recomputation, maximizing throughput by achieving optimal overlap between computation and transfer. We designed the KV distributor to dynamically allocate the proportion of tokens in the KV cache that are either loading (offloading) or recomputed, enhancing the overlap between computation and transfer by adjusting the recomputation ratio. The optimal ratio is determined through an integer linear programming (ILP) problem, solved with open-source mathematical programming solver [36].

The KV distributor dynamically balances transfer and computational overhead in LLM inference by integrating KV caching with selective recomputation without loss of precision. During decoding, the KV cache saved previous attention data to avoid redundant computations, but offloading incurs significant transfer overhead of KV cache transfer, making the loading and offloading process of KV cache a potential inference bottleneck. To mitigate this, the KV distributor reduces bottlenecks and increases throughput by replacing part of the transfer overhead with selective recomputation on the GPU with the same precision as the original KV cache. As the sequence length increases, the computation time becomes significantly lower than the transfer time, and this gap widens, making it challenging to mitigate the increased transfer overhead. Since the prefill phase only involves offloading the KV cache, while the decoding phase requires both offloading and loading, we detail our design for each phase separately.

Decoding Phase. In the decoding phase, which generates only one token's KV cache at a time, the primary transfer

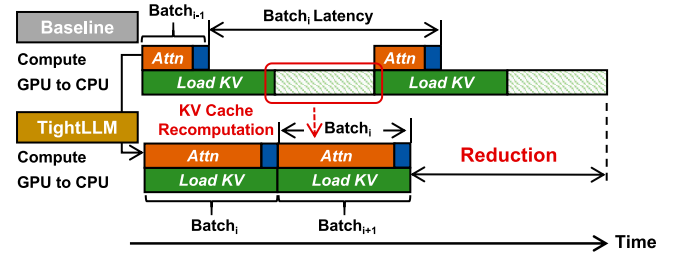


Fig. 6. Decoding latency with the KV distributor.

bottleneck arises from loading the existing KV cache. To balance computation time and KV cache transfer, the system dynamically adjusts the number of tokens to be recomputed. After determining the required recomputation tokens, we first transfer the KV cache of the tokens at the front of the sequence, and then recompute the KV cache of the subsequent tokens based on this transferred data. This adaptive approach ensures a more efficient balance between computation and transfer overhead, preventing bottlenecks and significantly boosting throughput during inference.

As shown in Fig. 6, during the decoding phase, the KV distributor's adjustment of specific tokens for recomputation drastically reduces latency. Before this adjustment, computation stalls due to high transfer overhead from loading the KV cache, leading to significant idle computational time and increased latency for the batch i . To address this, the KV distributor decreases the KV cache loading by KV cache recomputation, ensuring complete overlap between computation and transfer by controlling the number of tokens recomputed. This policy increases the computation time for batch $i - 1$ while reducing KV cache loading time for batch i , facilitating overlap and significantly enhancing overall throughput. To dynamically find the optimal number of recomputed tokens, we carry out the following mathematical modeling.

Optimal Recomputation Model. To achieve optimal performance under varying workloads, such as different sequence lengths, we developed a mathematical model that dynamically determines the optimal number of recomputed tokens in each iteration. We first model the computation and transfer times separately, and then define the optimization objective. The computation time consists of the attention layer time, $T_{compute}^{Attn}$, and the FFN layer time, $T_{compute}^{FFN}$, while the transfer time is represented by the KV cache loading time, T_{cache} . The optimization goal is maximizing system throughput, G_{token} as follows:

$$\max G_{token} = \max \frac{b_s m}{\sum_{i=1}^m \sum_{j=1}^n T_{batch}^{(i,j)}} \quad (1)$$

which is defined as the number of tokens generated divided by the total per-batch latency over m iterations of n layers with the batch size b_s . Therefore, for each inference, the problem is equivalent to minimizing the per-batch latency, T_{batch} , as follows:

$$\min T_{batch} = \min \{ \max \{ T_{compute}^{Attn} + T_{compute}^{FFN}, T_{cache} \} \} \quad (2)$$

When performing a batch in one layer of inference, the computation time is primarily the sum of the computation times for the attention and FFN layer [25], [37]. First, we define the number of tokens to be recomputed as N_r . The computation of the attention layer consists of three parts: QKV, mask, and project. **QKV** refers to the linear layer in multi-head self-attention that projects the input into queries, keys, and values, with FLOPs given by: $b_s N_r 2d_{model} 3d_{attn}$, where d_{model} is the dimensionality of the residual stream, and d_{attn} is the dimensionality of the output in attention. **Mask** involves the dot-product between queries and keys, with FLOPs given by: $b_s N_r 2d_{attn}(n_{ctx} + N_r)$, where n_{ctx} is the sequence length. **Project** is the linear layer that projects the concatenated attention head outputs back to d_{model} , with FLOPs given by: $b_s N_r 2d_{attn} d_{model}$. Thus, the total FLOPs of the attention layer are: $b_s N_r 2d_{attn} (4d_{model} + (n_{ctx} + N_r))$. Additionally, recomputing the KV cache before generating a new token incurs extra CPU overhead due to the need for activation passing and synchronization. To address this, we combine multiple token's recomputation into a single decoding computation to generate new tokens. Therefore, the FFN layer consists of two linear layers, with the total FLOPs given by: $b_s N_r 4d_{model} d_{FFN}$, where d_{FFN} is the size of the output dimensionality of the first linear layer.

Since hardware utilization can fluctuate dynamically at runtime, we define parameters, MFU (Model FLOPs Utilization), for calculating the self-attention layer, MFU_{Attn} , and the FFN layer, MFU_{FFN} , which must be derived from profiling⁵. Therefore, given the $FLOPS$ as the peak computational power of hardware, the computation time for the Attention and FFN layer can be expressed as:

$$T_{compute}^{Attn} = \frac{b_s N_r 2d_{attn} (4d_{model} + (n_{ctx} + N_r))}{FLOPS \times MFU_{Attn}} \quad (3)$$

$$T_{compute}^{FFN} = \frac{b_s N_r 4d_{model} d_{FFN}}{FLOPS \times MFU_{FFN}} \quad (4)$$

When offloading the KV cache, we segment it by the number of heads in the multi-head attention mechanism. The time required to load the KV cache into GPU HBM from CPU memory and disk consists of two components: reading the KV cache from disk and reading it from CPU memory. We define the proportion of the KV cache stored on disk as C_d and the proportion stored in CPU memory as C_c . The KV cache on the disk must first be loaded into CPU memory, and then both the disk and CPU-stored KV cache are loaded into GPU HBM. Therefore, the total KV cache loading time can be expressed as follows:

$$\begin{aligned} T_{cache} &= T_{cache}^{cpu} + T_{cache}^{disk} \\ &= \frac{4(C_d + C_c)b_s(N_{ctx} - N_r)d_{model}}{Bdw_{cg}} \\ &\quad + \frac{4C_d b_s(N_{ctx} - N_r)d_{model}}{Bdw_{dc}} \end{aligned} \quad (5)$$

⁵The MFU and bandwidth profiles are offline, so the overhead can be considered negligible.

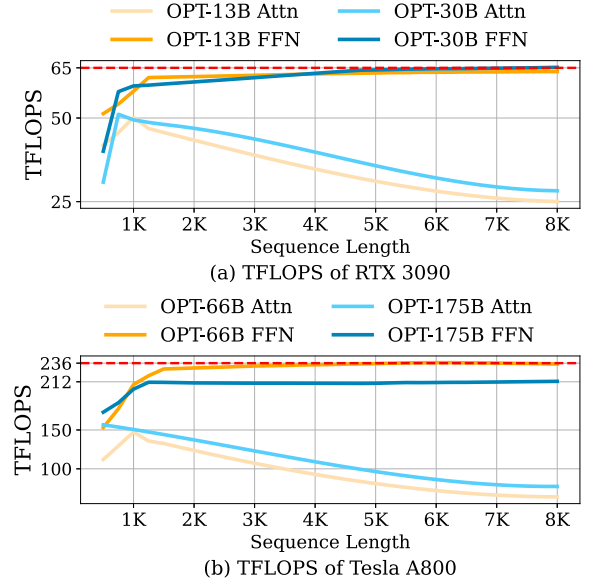


Fig. 7. GPU FLOPS varies with sequence length during computation.

where Bdw_{cg} is the PCIe bandwidth for loading from CPU memory to GPU memory, and Bdw_{dc} is the PCIe bandwidth for loading from disk to CPU memory.

Additionally, MFU varies with sequence length during computation. To analyze this, we measure the actual FLOPS for the attention and FFN layer computations across four models using the RTX 3090 and Tesla A800 GPUs, as the sequence length changes. As shown in Fig. 7, the FLOPS for the FFN layer increases sharply before stabilizing at longer sequence lengths. In contrast, the attention layer experiences a steep rise in FLOPS at shorter sequence lengths, but as the sequence length increases, the FLOPS gradually decreases due to the growing impact of memory access overhead. Given that the hardware's achievable FLOPS vary with sequence length, we approximate the determination of the optimal number of recomputed tokens as an integer programming problem. This problem can be efficiently solved using existing solvers [36], which handle the optimization with minimal computational overhead, ensuring that the impact on overall throughput is negligible.

Prefill Phase. To maximize the throughput, the KV distributor does not save the KV cache of tokens that need to be recomputed in the decoding phase during the prefill phase. During prefill, KV caches are generated for multiple tokens simultaneously, and there is no KV cache load; thus, the transfer bottleneck mainly arises from offloading generated KV caches. During inference, GPU memory usage increases with batch size, and the offloading system maintains the maximum batch size within limited memory⁶ to optimize throughput. However, as GPU memory is occupied by the previous layer's KV caches, the next layer's caches can only be offloaded after completely offloading the previous layer's caches. This dependency blocks prefill computation for the next layer, resulting in high latency.

⁶To ensure computation and transfer overlap, it typically uses half of the GPU's memory.

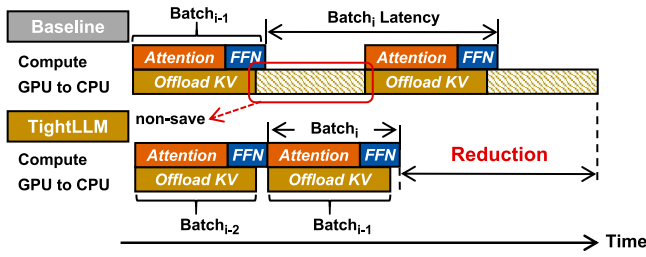


Fig. 8. Prefill latency with the KV distributor.

Since some tokens' KV caches are recomputed, offloading these caches from GPU to CPU during the prefill phase is unnecessary, reducing the time required for KV cache offloading.

As shown in Fig. 8, the offloading of the KV cache for batch $i - 1$ overlaps with the computation of batch i . However, until offloading is complete, the GPU cannot store three copies of the KV cache simultaneously, blocking the execution of batch $i + 1$. When part of the KV cache is recomputed, that portion can be discarded without being offloaded, which reduces latency and increases throughput in the prefill phase.

C. Weight Loader

Insight and Approach. To address the transfer bottleneck caused by loading weights, TightLLM pipelines weight loading to conceal transfer overhead across multiple batches of computation. Since the weights of the transformer blocks need to be loaded only once for inference over multiple batches, TightLLM slices each transformer block, allowing the system to pipeline the loaded weights. Specifically, during the execution of each batch in the *current* transformer block, TightLLM loads a portion of weight for the *next* transformer block, which incurs relatively minor overhead. By doing so, TightLLM is able to facilitate overlap between the transfer time of loading weight and the GPU computation. This policy alleviates the latency bottleneck by ensuring that weight loading does not block computation, thereby improving overall throughput. As the transfer overhead composition differs between the decoding and prefill phases, we will discuss them separately.

Decoding Phase. To mitigate the high overhead of loading weights during the decoding phase, we slice the weights and pipeline the loads to overlap with computations across multiple batches. First, we slice the weights based on the number of batches over which they are amortized. Alongside each batch inference, we load the sliced weight segments and the KV cache for the next batch. Once all weight slices are loaded into the GPU, we synchronize them before proceeding with inference in the next transformer block. By significantly reducing the loading latency of the weight block through slicing, we effectively hide this latency from computation, thereby lowering overall latency and increasing throughput.

Short Sequence Length. Pipeline weight loading can significantly reduce latency during the decode phases, especially when the sequence length is short. As shown in Fig. 9(a), switching transformer blocks incurs substantial transfer overhead from loading weights, severely increasing the latency of the first

batch due to this transfer bottleneck. By amortizing the weight overhead across four batches, we slice the weights into four segments and load the corresponding weight slices for each batch inference. This approach greatly reduces the weight overhead, ensuring that the inference latency for each batch surpasses the latency of weight loading, thereby preventing transfer from becoming a bottleneck. Additionally, the latency for the first batch is significantly reduced, enhancing overall throughput.

Long Sequence Length. However, as the sequence length grows, the KV cache loading overhead gradually increases, making it challenging to fully cover the transfer overhead even with all batches in a block. In such cases, it is necessary to adjust KV recomputation while considering weight loading. As shown in Fig. 9(b), the total transfer time for both Batch 1 and Batch 2 exceeds the computation latency with sliced weights. Although the weight slices for both batches were loaded in approximately the same amount of time, we increased the KV recomputation for Batch 1 and reduced it for Batch 2. This adjustment balanced the transfer and computation within each batch, minimizing GPU idle time and reducing token generation time, ultimately enhancing throughput. If the batch consists of sequences of mixed lengths, we dynamically handle the recomputation of the KV cache and weight slices based on the execution time of the sequences. Specifically, when the computation time of a mixed batch is less than the transfer time required for the current weight slice, we adopt a similar strategy as for longer sequences, i.e., increase the amount of KV cache recomputation.

Update Recomputation Model. To achieve maximum throughput for different sequence lengths, we need to update the model from the previous subsection Section IV-B to account for long sequence cases involving model weight loading. First, the loading latency should be the sum of the KV cache loading, the attention weights loading, and the FFN weights loading. Next, we will model the loading latency for the weights of the newly added attention layer and the FFN weights. Thus, for each inference, the problem is equivalent to minimizing the per-batch latency, T_{batch} , as follows:

$$\min\{\max\{T_{compute}^{Attn} + T_{compute}^{FFN}, T_{cache} + T_{weight}^{FFN} + T_{weight}^{Attn}\}\} \quad (6)$$

Next, we model in detail the loading time for the attention and FFN layers. The latency for data loading to the GPU for both the Attention layer and the FFN layer can be modeled mathematically. For the Attention layer, the latency T_{weight}^{Attn} is given by $T_{weight}^{Attn} = \frac{1}{b_n} \times \left(\frac{(W_d + W_c) \cdot d_{model} \cdot 4d_{attn}}{Bdw_{cg}} + \frac{W_d \cdot d_{model} \cdot 4d_{attn}}{Bdw_{dc}} \right)$, where W_d is the proportion of the model weight stored on disk and W_c is the proportion stored in CPU memory. Therefore, the loading time of the slice weight is as follows:

$$T_{weight}^{Attn} = \frac{4d_{model}d_{attn}}{b_n} \times \left(\frac{W_d + W_c}{Bdw_{cg}} + \frac{W_d}{Bdw_{dc}} \right) \quad (7)$$

where b_n is the number of batches to amortize the weight loading I/O overhead during inference. Similarly, the FFN layer

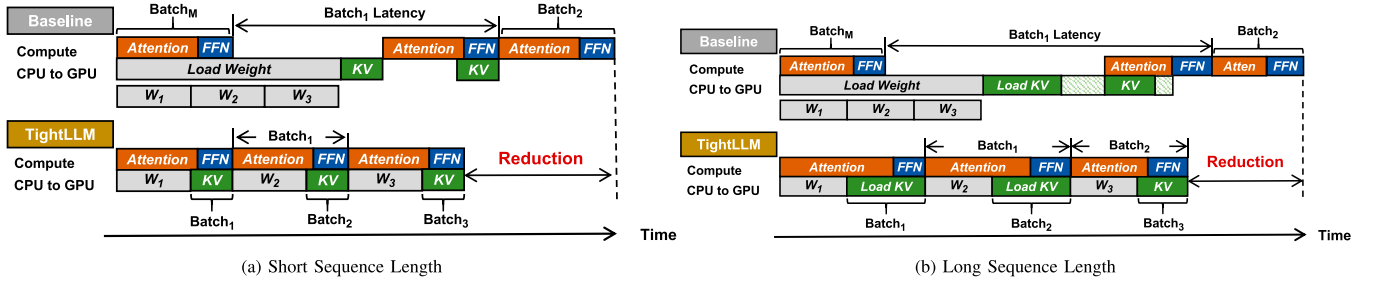


Fig. 9. Decoding latency with the pipeline weight loader.

loading latency is as follows:

$$T_{weight}^{FFN} = \frac{2d_{model}d_{mlp}}{b_n} \times \left(\frac{W_d + W_c}{Bdw_{cg}} + \frac{W_d}{Bdw_{dc}} \right) \quad (8)$$

where d_{mlp} is the dimension of the Multi-Layer Perceptron. Both equations show that the added parameter weights and model size are known. Therefore, the problem is solved in the same way and can still be solved by the planning solver.

Prefill Phase. In the prefill phase, the pipeline loads weights similarly to the decoding stage. However, this phase can operate in parallel by loading weight and offloading the KV cache, resulting in two potential transfer bottlenecks: (1) if the overhead of offloading the KV cache exceeds that of loading weight, the bottleneck is offloading the KV cache; (2) if the overhead of loading weight is greater, then the bottleneck shifts to loading weight. As the weight loading is fixed, the source of bottlenecks in the prefill phase depends on the length of the prompt inputs. With longer prompts, more KV caches must be offloaded, making offloading the KV caches the primary bottleneck. Conversely, shorter prompts lead to bottlenecks in loading weight. Consequently, TightLLM addresses both KV cache offloading and weight loading when optimizing transfer overhead in the prefill phase.

D. Put It All Together

Based on the KV distributor and weight loader, we propose an adaptive offloading algorithm as shown in Algorithm 1. The Adaptive Offloading Algorithm optimizes the processing of requests in an LLM inference system by dynamically managing the transfer and computation of KV cache and weight based on the current state S and the model config M . The algorithm initializes the system with specified inference models and tasks, then processes requests in a loop until all are handled (lines 1-3). For each request, it calculates the average sequence length $AvgSeqLen$ (line 4), the transfer time of KV cache T_{cache} and weight T_{weight} (line 5-6), and computes time $T_{compute}$ (line 7). It also assesses the tokens' number of required recomputation N_r (line 8) to optimize performance.

The algorithm then iterates through each layer of the model (line 9) and processes them in batches (line 10). For the first batch, it loads the weight slice for the next layer (lines 11-13). In the decoding phase, it loads the cache for the next batch and computes based on the N_r (lines 14-18). In the prefill phase, it computes for the current batch and offloads the cache for

Algorithm 1: Adaptive Offloading Algorithm.

Data: Offloading State: S , Model Config: M

```

1 Collection of the model parameters of all layers:  $L$ 
2 Collection of all batches composed by inference requests:  $B$ 
3 while  $Requests \neq NULL$  do
4    $AvgSeqLen \leftarrow Get\_Sequence\_Length(Requests)$ ;
5    $T_{cache} \leftarrow Get\_Cache\_Loading(S, M, SeqLen)$ ;
6    $T_{weight} \leftarrow Get\_Weight\_Loading(S, M, B)$ ;
7    $T_{compute} \leftarrow Get\_Compute(M, SeqLen)$ ;
8    $N_r \leftarrow Get\_Recompute(M, T_{compute}, T_{cache}, T_{weight}, AvgSeqLen)$ ;
9   for  $layer$  in  $L$  do
10    for  $batch$  in  $B$  do
11      if  $Batch.Index = First$  then
12        Loading_Weight_Slice(layer.next, batch);
13      end
14      if  $S.Phase = DECODING$  then
15        Loading_Cache(layer, batch.next);
16        Compute(layer, batch,  $N_r$ );
17        Offloading_Cache(layer, batch.prev,  $N_r$ );
18      end
19      if  $S.Phase = PREFILL$  then
20        Compute(layer, batch);
21        Offloading_Cache(layer, batch.prev,  $N_r$ );
22      end
23      Activation_Related(layer, batch.next);
24    end
25    Post_Processing();
26  end
27 end
```

the previous batch (lines 19-22). Activation-related operations are then handled for the next batch (line 23). Finally, post-processing steps are executed after processing all batches within a layer (line 25), enabling efficient management of transfer and computational resources tailored to the request being processed.

V. EVALUATION

We evaluate TightLLM's throughput by comparing it with existing offloading-based LLM inference systems. Additionally, we perform an ablation study to assess how TightLLM's design choices influence throughput optimization.

A. Experiment Setting

Implementation. We implement all features of TightLLM described in Section IV in roughly 5k lines of code. The entire

TABLE I
HARDWARE AND SOFTWARE SPECIFICATIONS

Hardware	Consumer-Grade	Server-Grade
CPU	24 CPU cores	26 CPU cores
GPU	NVIDIA RTX 3090	NVIDIA Tesla A800
DRAM	DDR4 32GB $\times 4$	DDR4 64GB $\times 4$
NVMe	3.84 TB $\times 2$	7.68T $\times 1$
S/W	PyTorch 2.2.1 CUDA 11.7	PyTorch 2.1.2 CUDA 12.2

inference process is built using PyTorch [38]. Additionally, TightLLM utilizes multiple CUDA streams to concurrently execute computation and data transfer processes.

Hardware. We conducted experiments using two hardware configurations: a consumer-grade machine C_1 equipped with the RTX 3090 GPU and a server-grade machine C_2 featuring the Tesla A800 GPU, as detailed in Table I. Although our methods and implementations are not tied to any specific hardware architecture, certain configurations, such as unified memory or higher bandwidth, may enhance the effectiveness of our approach.

Model. TightLLM works well for any large transformer-based language model, and we have utilized two popular existing LLM architectures, OPT [34] and LLaMA [13], for our experiments. Specifically, we evaluated four OPT models with parameter sizes ranging from 13B to 175B, as well as the Llama-2-13B and Llama-2-70B models. We tested models with fewer parameters (OPT-13B, OPT-30B and Llama-13B) on machine C_1 and those with larger parameters (OPT-66B, OPT-175B and Llama-70B) on machine C_2 .

Workload. We tested prompt lengths ranging from 0.5k to 8k, utilizing the maximum batch size supported by each machine. To evaluate the effectiveness of various methods on a single GPU with limited CPU memory, we compared token generation throughput across scenarios with a generated length of 32 tokens per prompt. Additionally, to demonstrate that TightLLM significantly reduces the completion time for large-scale tasks, we measured the total completion time across multiple datasets: translation, text classification, feature extraction, and summarization.

Baseline. We use Hugging Face Accelerate [24] and FlexGen [25] as baselines, as both are open-source tools that support offloading-based LLM inference, including the offloading of model weight parameters. These systems are designed to maximize throughput under limited GPU memory conditions. Accelerate enables partial weight offloading by updating layer weights after each layer's computation. FlexGen supports offloading both model weights and the KV cache, processing multiple batches per layer to reduce the times of weight loading during LLM inference.

Metrics. We focused on three key metrics: (1) *Throughput*: the number of tokens processed in the prefill phase or generated in the decoding phase per unit of time. Throughput in the prefill phase refers to the number of input tokens the model can process, reflecting its ability to preprocess input efficiently. Throughput in the decoding phase indicates the number of output tokens generated per unit of time, showcasing the model's

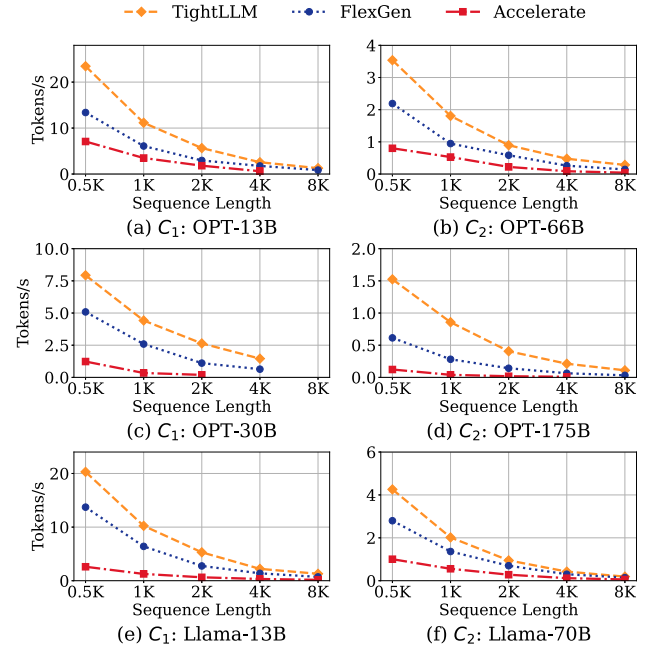


Fig. 10. Decode phase throughput.

efficiency in text generation. (2) *Per-token Latency*: To illustrate how TightLLM enhances throughput, we present the latency composition for each token processed or generated. The latency composition primarily consists of GPU computation time and transfer time, where transfer time specifically refers to the duration that is not hidden by computation. (3) *Task Completion Time*: To demonstrate the real-world benefits of high throughput in TightLLM, we record the total completion time for large-scale tasks. The total completion time for a large-scale task is the sum of the completion time of all its sub-tasks. For example, in a literature translation task, its completion time is equal to the duration required to translate all pieces of literature involved.

B. Comparison Results

In this section, we show the superior performance by comparing the key metrics of TightLLM and alternative systems.

Decoding phase. Fig. 10 illustrates the token generation throughput during the decoding phase for TightLLM and two baseline alternatives, showing that TightLLM consistently achieves higher throughput. Specifically, TightLLM improves throughput by a factor of $1.3\times$ to $23\times$ compared to FlexGen and Accelerate, highlighting the effectiveness of its adaptive offloading policy. For the different LLM architectures, OPT and LLaMA, TightLLM similarly enhances throughput, achieving increases of over 3 times at maximum.

TightLLM exhibits varying enhancement capabilities based on different configurations and sequence lengths. (1) *Different configurations*. On machine C_1 , TightLLM can improve the average throughput of the LLM inference by up to $10.4\times$, while on machine C_2 , this improvement reaches up to $19.9\times$. The greater transfer bottleneck on machine C_2 is due to its ability to deploy larger models and transfer more data between the CPU and GPU. Consequently, the benefits of TightLLM's transfer

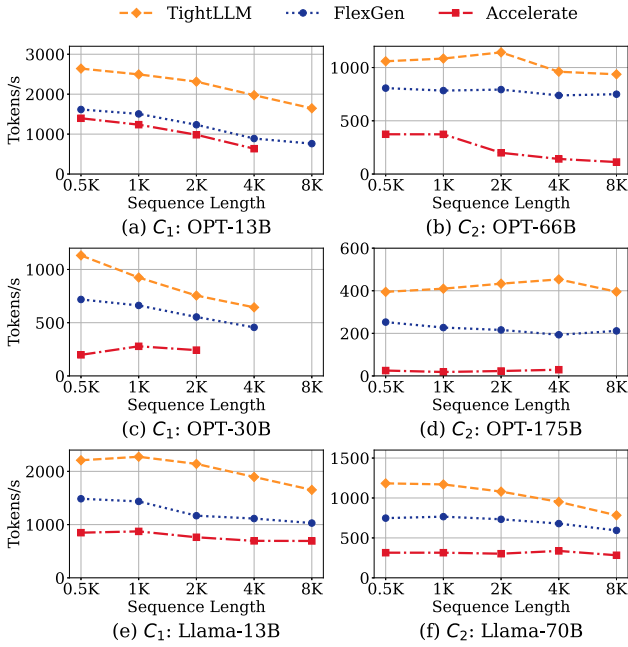


Fig. 11. Prefill phase throughput.

bottleneck mitigation are more pronounced in this context. (2) *Sequence lengths*. For a sequence length of 0.5k, TightLLM maximizes throughput between $2.4\times$ and $12.4\times$. However, for a sequence length of 4k, the maximized throughput ranges from $3.3\times$ to $22.8\times$. This increase is attributed to the growing transfer bottleneck imposed by the KV cache as sequence length increases, which the comparison systems cannot alleviate. Thus, as the sequence length expands, TightLLM becomes increasingly effective in enhancing throughput performance.

The throughput improvement of TightLLM during the decoding phase can be attributed to its reduced per-token generation latency. As shown in Fig. 12(a), TightLLM reduces the latency for generating a single token by 59.7% to 95.7% compared to alternative systems when using the OPT-175 model. The KV cache loading policies in these alternative systems fail to effectively minimize transfer overhead. In contrast, TightLLM's KV distributor employs selective KV recomputation to alleviate transfer bottlenecks associated with KV cache loading, achieving the optimal recompute ratio to maximize throughput. Although TightLLM cannot entirely eliminate transfer latency, the gap is minimal. Compared to other systems, TightLLM's recomputation of the KV cache increases computation time by $3.1\times$ to $5.4\times$, but reduces transfer time by 68.9% to 97.5%. This significantly reduces the time the GPU takes to generate each token. Furthermore, TightLLM's weight loader concurrently mitigates the transfer time associated with weight loading, significantly reducing the time required for token generation.

Prefill phase. Fig. 11 illustrates the token generation throughput during the prefill phase for TightLLM and two baseline alternatives. TightLLM consistently achieves higher throughput, with improvements ranging from 1.2 to 22 times compared to the alternatives. Unlike the decoding phase, throughput in the

prefill phase does not vary steadily with sequence length or inference configuration. This inconsistency arises from dynamically changing transfer bottlenecks. When model parameters are large and the sequence is short, the primary bottleneck is KV cache offloading. Conversely, when model parameters are small and the sequence is long, weight loading becomes the limiting factor. Consequently, TightLLM's performance advantage fluctuates significantly due to the complexity of the transfer bottleneck as it interacts with these two variables.

Similar to the decoding phase, the increase in throughput during the prefill phase is driven by a reduction in processing time per token. As shown in Fig. 12(b), the inference latency of TightLLM is significantly reduced by 35.9% to 95.5% due to decreased transfer overhead. Unlike the decoding phase, there is no KV cache recomputation during the prefill phase, so the computation time for TightLLM and the alternative systems remains essentially the same. However, TightLLM reduces transfer time by 64.6% to 99.1%. This reduction in transfer overhead results from two key factors, similar to those in the decoding phase: (1) The KV distributor minimizes the percentage of KV cache offloaded to CPU memory based on the recomputation percentage from the decoding phase, thereby decreasing transfer overhead and enhancing throughput. (2) TightLLM's weight loader mitigates transfer overhead related to weight loading by concealing the substantial overhead within the computation of multiple batches.

Since Accelerate does not incorporate an offloading strategy to alleviate transfer overhead, its transfer overhead represents a larger portion of the overall inference, resulting in reduced throughput. While FlexGen addresses some transfer overhead through overlapping computation with transfer and employing multi-batch inference strategies, as discussed in Section II, it still experiences significant transfer time and lower throughput due to its inability to adapt to dynamic transfer changes. Moreover, both TightLLM and its alternatives encounter out-of-memory (OOM) issues on the GPU when handling longer sequences and larger models. However, Accelerate is more vulnerable to OOM issues, since it only offloads weight to the CPU while keeping all KV cache in GPU memory.

Large-scale tasks. To verify that TightLLM reduces the total completion time of large-scale tasks by enhancing throughput in both the prefill and decoding phases, we compared the completion times across four datasets for TightLLM and two alternative systems. These datasets consist of numerous subtasks, and the overall task is considered complete only when all subtasks have finished processing. Consequently, our focus is on the completion time of the tasks rather than the inference latency of individual subtasks. As shown in Fig. 13, we define the completion time for TightLLM as 1 for normalization. Across the four datasets, TightLLM achieves a reduction in completion time by 59.6%-94.9%. This demonstrates that TightLLM significantly decreases task execution time for large-scale tasks and exhibits consistent performance.

Since performance differences arise from various factors, we hypothesize that the advantages of TightLLM stem from several key elements. In Section V-C, we present a detailed

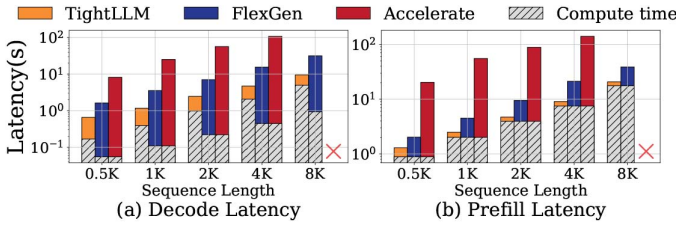


Fig. 12. Inference latency composition for OPT-175B in C_2 machines: transfer and computation time.

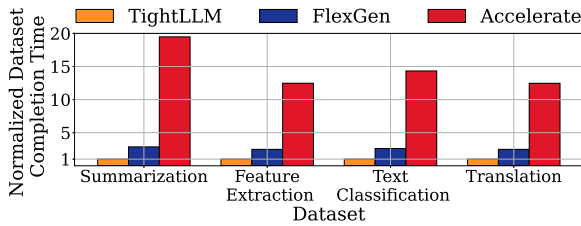


Fig. 13. Large-scale tasks completion time.

ablation study to quantify the impact of each design choice on throughput improvement.

C. Ablation Study

In this section, we disable each of TightLLM's novel features to demonstrate its contribution to guaranteeing high throughput. **Ablation in decoding phase.** TightLLM outperforms alternative systems primarily due to its KV distributor, which significantly alleviates transfer bottlenecks caused by KV cache loading in the decoding phase. To verify this, we compare TightLLM without the KV distributor, relying solely on loading the KV cache when previous KV matrices are needed. As shown in Fig. 14, this setup demonstrates the throughput with all optimization components enabled versus the normalized throughput without the KV distributor in the decoding phase. When TightLLM does not utilize the KV distributor, the throughput in the decoding phase decreases by 12.6% to 43.1%. Furthermore, TightLLM's weight loader also contributes to its performance by alleviating transfer bottlenecks associated with weight loading. To validate this, we compare TightLLM without the weight loader, which loads a transformer block into the GPU in a single transfer. As illustrated in Fig. 14, omitting the weight loader results in a throughput reduction of 4.4% to 74.8% in the decoding phase.

Ablation in prefill phase. The superior performance of TightLLM over alternative systems can be attributed largely to its KV distributor, which effectively reduces transfer bottlenecks linked to KV cache offloading. To demonstrate this, we conducted a comparison of TightLLM without the KV distributor, which offloads the KV cache for every token during the prefill phase. As illustrated in Fig. 15, it highlights the throughput when all optimization components are active versus the normalized throughput without the KV distributor in the prefill phase. Omitting the KV allocator leads to a significant throughput reduction of up to 46.6% in this phase. Moreover,

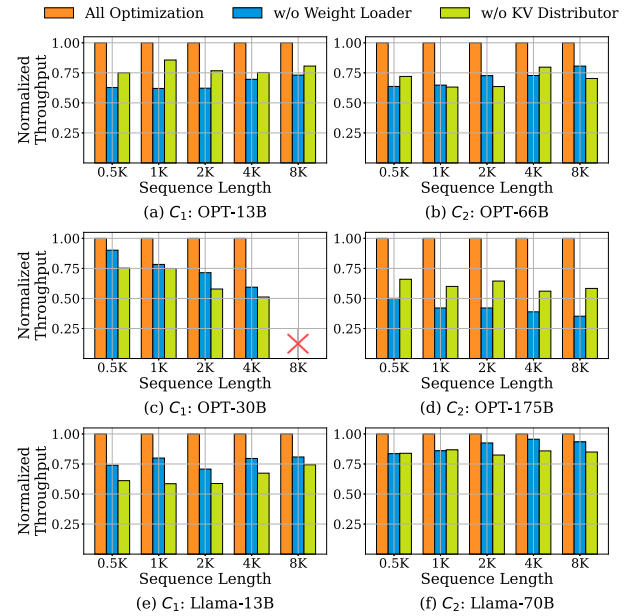


Fig. 14. Ablation study of decoding phase throughput.

TightLLM's weight loader plays a crucial role in enhancing performance by mitigating transfer bottlenecks related to weight loading. The data in Fig. 15 shows that when the weight loader is not utilized, the throughput can drop by as much as 38.2% during the prefill phase.

VI. RELATED WORK

Inference Offloading. Existing works [23], [24], [25], [26], [35], [39], [40] typically use offloading to enable an LLM inference with a limited GPU memory device. HuggingFace Accelerate [24] and DeepSpeed [23] offload the entire model weight in CPU memory and load the weight layer-by-layer into the GPU for inference computation, leveraging prefetching to hide some transfer overhead of loading weight. FlexGen [25] supports partial offloading for low-latency scenarios by utilizing more GPU memory for weights, and aggressively offloads all tensors in high-throughput scenarios to achieve larger inference batch size and numbers. Other works [26], [35], [41] focus on leveraging the CPU computation to increase the throughput when offloading. PowerInfer [39] and LLM in a flash [40] offload most of the inactive weight to the CPU memory based on the sparsity in FFN computation. Particularly, InfiniGen [37] offloads the full KV cache to CPU memory and decreases the transfer time by sparsely loading the KV cache. These systems cannot eliminate the large transfer overhead of offloading without loss of precision, whereas TightLLM dynamically adjusts the computation and transfer ratio, including KV cache management and weight loading, based on real-time computational requirements and transfer overhead.

KV Cache Management. There are some research that explore the KV cache management to reduce the KV cache size and mitigate bottlenecks from it. vLLM [29] mitigates the KV cache memory waste from fragmentation and duplication. SGLang [42] shares prompt prefixed to reduce the KV computation for

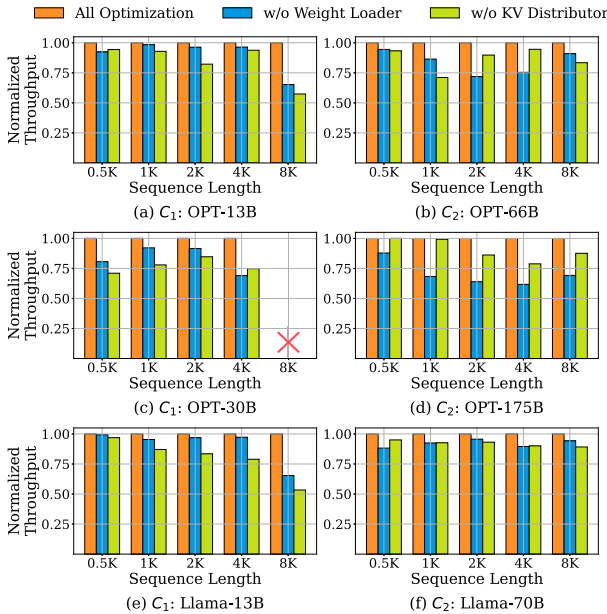


Fig. 15. Ablation study of prefill phase throughput.

the tasks share the same prefixed. CachedAttention [43] and HCache [44] offload the KV cache to the CPU memory and load them on demand for multi-turn conversations. In addition, some work [21], [37] exploits the sparsity of the KV cache to reduce the size of the KV cache. However, these tasks either do not fully utilize the computational power of the GPU or incur some loss of inference precision.

Other Memory Optimizations. Besides, numerous systems propose model compression and low-level optimization methods to reduce the GPU memory usage when LLM inference. To reduce the storage overhead, existing work employs quantization and compression techniques on both model weight [18], [19] and KV cache [20], [21]. Flashattention [45] uses a fused attention kernel to reduce memory usage by system optimization. These memory optimization methods and TightLLM are orthogonal to each other and can be combined to achieve smaller GPU memory usage or improve higher throughput.

VII. DISCUSSION

Token-level Windowing Mechanism in Offloading. Attention requires loading all relevant KV cache data into GPU memory simultaneously to ensure each input token has access to the full context. Consequently, the smallest unit for KV cache loading is the entire layer, not smaller data windows. While sequence parallelism [17] allows token-level distribution of attention across multiple machines, splitting attention at the token level on a single GPU reduces computational efficiency and may decrease throughput. We plan to explore this approach in future work to achieve a finer-grained balance between computation and data transfer.

Compression and Quantization. TightLLM runs independently of compression techniques, but its adaptive strategies can be further optimized to improve both memory efficiency and throughput. By incorporating compression

methods, TightLLM’s performance may be enhanced, enabling it to handle larger models or batch sizes on hardware with more limited resources. Investigating the interaction between compression techniques and TightLLM’s adaptive mechanisms represents a promising avenue for future research.

Runtime Scheduling for Offloading Inference. While runtime dynamic offloading allows for fine-grained control over transfer and computation, it also introduces additional overhead, which may not always surpass model-based approaches. The effectiveness of these strategies depends on factors like the frequency of runtime information collection, parameter synchronization, and other related overheads. Consequently, there is significant potential for future research to identify more efficient methods for achieving an optimal balance.

VIII. CONCLUSION

In this paper, we present TightLLM, an adaptive offloading-based inference system for large language models designed to maximize throughput on hardware with limited GPU memory. TightLLM introduces two key innovations: a KV distributor that employs a *trade-compute-for-transfer* strategy to dynamically recompute portions of the KV cache, thereby overlapping data transfer with computation, and a weight loader that slices model weights and distributes the loading process *across multiple batches* to amortize weight loading overhead. Unlike existing systems’ *static* offloading policies, which result in significant transfer overhead and GPU idleness, TightLLM’s *adaptive* approach effectively adjusts to varying transfer overheads, significantly enhancing inference throughput. Evaluation across various GPU hardware and LLM models demonstrates that TightLLM achieves 1.3 to 23 times higher throughput during the decoding phase and 1.2 to 22 times higher throughput in the prefill phase, when compared to state-of-the-art offloading systems. Furthermore, TightLLM reduces the completion latency for large-scale tasks by 59.6% to 94.9%, enabling faster processing and generation of substantial numbers of tokens.

REFERENCES

- [1] W. Yu et al., “A survey of knowledge-enhanced text generation,” *ACM Comput. Surveys*, vol. 54, no. 11s, pp. 1–38, 2022.
- [2] A. Rogers, M. Gardner, and I. Augenstein, “QA dataset explosion: A taxonomy of NLP resources for question answering and reading comprehension,” *ACM Comput. Surveys*, vol. 55, no. 10, pp. 1–45, 2023.
- [3] H. Xu, Y. J. Kim, A. Sharaf, and H. H. Awadalla, “A paradigm shift in machine translation: Boosting translation performance of large language models,” in *Proc. ICLR*, 2024.
- [4] B. Zhang, B. Haddow, and A. Birch, “Prompting large language model for machine translation: A case study,” in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2023, pp. 41092–41110.
- [5] M. Blott et al., “Evaluation of optimized CNNs on heterogeneous accelerators using a novel benchmarking approach,” *IEEE Trans. Comput.*, vol. 70, no. 10, pp. 1654–1669, Oct. 2021.
- [6] X. Chen et al., “Spreadsheetcoder: Formula prediction from semi-structured context,” in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 1661–1672.
- [7] OpenAI. Accessed: 2024. [Online]. Available: <https://openai.com/>
- [8] Anthropic. Accessed: 2024. [Online]. Available: <https://www.anthropic.com>
- [9] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly,” *High-Confidence Comput.*, 2024, Art. no. 100211.

- [10] S. Sadeepa, K. Kavinda, E. Hashika, C. Sandeepa, T. Gamage, and M. Liyanage, "DISLLM: Distributed LLMs for privacy assurance in resource-constrained environments," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Piscataway, NJ, USA: IEEE, 2024, pp. 1–9.
- [11] K. Cheng et al., "Private inference for deep neural networks: A secure, adaptive, and efficient realization," *IEEE Trans. Comput.*, vol. 72, no. 12, pp. 3519–3531, Dec. 2023.
- [12] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [13] H. Touvron et al., "Llama: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [14] NVIDIA, "FasterTransformer." Accessed: 2024. [Online]. Available: <https://github.com/NVIDIA/FasterTransformer>
- [15] L. Zheng et al., "Alpa: Automating inter- and {intra-operator} parallelism for distributed deep learning," in *Proc. OSDI*, 2022, pp. 559–578.
- [16] Z. Li et al., "{AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving," in *Proc. OSDI*, 2023, pp. 663–679.
- [17] B. Wu, S. Liu, Y. Zhong, P. Sun, X. Liu, and X. Jin, "LoongServe: Efficiently serving long-context large language models with elastic sequence parallelism," in *Proc. ACM SIGOPS 30th Symp. Operating Syst. Princ. (SOSP)*, 2024, pp. 640–654.
- [18] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "GPT3.5: 8-bit matrix multiplication for transformers at scale," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 30318–30332.
- [19] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 27168–27183.
- [20] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, "Model tells you what to discard: Adaptive KV cache compression for LLMs," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024.
- [21] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024.
- [22] C. Gong, Y. Chen, Y. Lu, T. Li, C. Hao, and D. Chen, "VecQ: Minimal loss DNN model compression with vectorized weight quantization," *IEEE Trans. Comput.*, vol. 70, no. 05, pp. 696–710, May 2021.
- [23] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 3505–3506.
- [24] S. Gugger et al., "Accelerate: Training and inference at scale made simple, efficient and adaptable," 2022. Accessed: 2024. [Online]. Available: <https://github.com/huggingface/accelerate>
- [25] Y. Sheng et al., "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2023, pp. 31094–31116.
- [26] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Efficient heterogeneous parallel inference for large language models on resource-constrained devices," in *Proc. MLSys*, vol. 6, 2024, pp. 162–172.
- [27] W. G. Choi, D. Kim, H. Roh, and S. Park, "OurRocks: Offloading disk scan directly to GPU in write-optimized database system," *IEEE Trans. Comput.*, vol. 70, no. 11, pp. 1831–1844, Nov. 2021.
- [28] S. Li et al., "NDRec: A near-data processing system for training large-scale recommendation models," *IEEE Trans. Comput.*, vol. 73, no. 05, pp. 1248–1261, May 2024.
- [29] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ.*, 2023, pp. 611–626.
- [30] M. Kang, J. Park, H. Shin, J. Shin, and L.-S. Kim, "ToEx: Accelerating generation stage of transformer-based language models via token-adaptive early exit," *IEEE Trans. Comput.*, vol. 73, no. 09, pp. 2248–2261, Sep. 2024.
- [31] L. Liu, Z. Qu, Z. Chen, F. Tu, Y. Ding, and Y. Xie, "Dynamic sparse attention for scalable transformer acceleration," *IEEE Trans. Comput.*, vol. 71, no. 12, pp. 3165–3178, Dec. 2022.
- [32] H. Wang et al., "Raptor-T: A fused and memory-efficient sparse transformer for long and variable-length sequences," *IEEE Trans. Comput.*, vol. 73, no. 07, pp. 1852–1865, Jul. 2024.
- [33] J. Kaplan et al., "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
- [34] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.
- [35] D. Park and B. Egger, "Improving throughput-oriented llm inference with CPU computations," in *Proc. PACT*, 2024, pp. 233–245.
- [36] Gurobi. Accessed: 2024. [Online]. Available: <https://www.gurobi.com>
- [37] W. Lee, J. Lee, J. Seo, and J. Sim, "{InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management," in *Proc. OSDI*, 2024, pp. 155–172.
- [38] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019.
- [39] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. ACM SIGOPS 30th Symp. Operating Syst. Princ. (SOSP)*, New York, NY, USA: ACM, 2024, pp. 590–606.
- [40] K. Alizadeh et al., "LLM in a flash: Efficient large language model inference with limited memory," 2023, *arXiv:2312.11514*.
- [41] A. Ghose, S. Singh, V. Kulaharia, L. Dokara, S. Maity, and S. Dey, "PySchedCL: Leveraging concurrency in heterogeneous data-parallel systems," *IEEE Trans. Comput.*, vol. 71, no. 09, pp. 2234–2247, Sep. 2022.
- [42] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," in *Proc. 38th Annu. Conf. Neural Inf. Process. Syst.*, 2024, pp. 62557–62583.
- [43] B. Gao et al., "{Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2024, pp. 111–126.
- [44] S. Gao, Y. Chen, and J. Shu, "Fast state restoration in LLM serving with hcache," in *Proc. EuroSys*, 2025, pp. 128–143.
- [45] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 16344–16359.



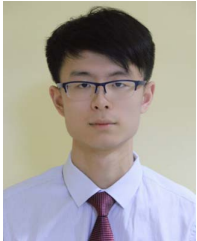
Yitao Hu received the B.S. degree from the Department of Electrical Engineering, Shanghai Jiao Tong University, China, in 2014, and the Ph.D. degree from the Networked Systems Lab (NSL), University of Southern California, USA, in 2021. Currently, he is an Assistant Professor with the College of Intelligence and Computing, Tianjin University, China. His research focuses on developing inference systems capable of deploying large language model (LLM) and deep neural networks (DNN) models in large-scale cloud clusters, aiming for peak performance, efficiency, and scalability. He has published papers at leading conferences/journals, including ACM Symposium on Cloud Computing (SoCC); ACM International Conference on Ubiquitous Computing (Ubicomp); IEEE International Conference on Computer Communications (INFOCOM); IEEE/ACM International Symposium on Quality of Service (IWQoS); ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS); ACM Special Interest Group on Data Communication (SIGCOMM), and IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS. He received the Best Paper Award from SoCC 2024.



Xiulong Liu (Senior Member, IEEE) received the B.E. and Ph.D. degrees from Dalian University of Technology, China, in 2010 and 2016, respectively. Currently, he is a Professor with the College of Intelligence and Computing, Tianjin University, China. He also worked as a Visiting Researcher with Aizu University, Japan; a Postdoctoral Researcher with HK PolyU, Hong Kong; and a Postdoctoral Researcher with the School of Computing Science, Simon Fraser University, Canada. His research papers were published in many prestigious journals and conferences, including IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE International Conference on Network Protocols (ICNP), etc. His research interests include wireless sensing and communication, indoor localization, and networking. He received the Best Paper Awards from International Conference on Algorithms and Architectures for Parallel Processing (ICA3PP) 2014 and IEEE SYSTEMS JOURNAL 2017.



Guotao Yang received the B.E. degree in software engineering in 2023 from Tianjin University, Tianjin, China, where he is currently working toward the master's degree in computer science and technology. His research interests include cloud computing and distributed systems. Specifically, his research focuses on AI training and inference systems.



Linxuan Li received the B.E. degree in computer science and technology from Chongqing University, Chongqing, China, in 2022. He is currently working toward the master's degree in computer science and technology from Tianjin University, Tianjin, China. His research interests include infrastructure supporting large language models (LLMs) and distributed systems for LLMs.



Kai Zeng is currently working toward the bachelor's degree in artificial intelligence with the College of Intelligence and Computing, Tianjin University. His research interests include cloud computing, distributed systems, and deep learning.



Zhixin Zhao (Graduate Student Member, IEEE) received the B.E. degree in computer science and technology from the Northwest A&F University, Shaanxi, China, in 2022. He is currently working toward the Ph.D. degree in software engineering from Tianjin University, Tianjin, China. His research interests include machine learning systems and cloud computing. In particular, in machine learning systems, his research focuses on inference and deployment optimization for deep neural networks (DNN) models and large language models (LLMs).



Sheng Chen (Member, IEEE) received the bachelor's degree from Dalian Maritime University, in 2015, the master's degree from Dalian University of Technology, in 2017, and the Ph.D. degree, in 2023. Currently, he is a Postdoctoral Researcher with the College of Intelligence and Computing, Tianjin University, China. His research interests include data center networks, edge computing, wireless sensing, and indoor localization.



Laiping Zhao (Member, IEEE) received the B.S. and M.S. degrees from Dalian University of Technology, China, in 2007 and 2009, respectively, and the Ph.D. degree from the Department of Informatics, Kyushu University, Fukuoka, Japan, in 2012. Currently, he is a Professor with the College of Intelligence and Computing, Tianjin University, Tianjin, China. His research interests include cloud computing and serverless computing. He has authored or co-authored more than 70 papers in international journals and conferences, including IEEE

TRANSACTIONS ON CLOUD COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON SERVICE COMPUTING, ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS); International Conference for High Performance Computing, Networking, Storage, and Analysis (SC); USENIX Annual Technical Conference (ATC); European Conference on Computer Systems (EuroSys), etc.



Wenxin Li received the B.E. and Ph.D. degrees from Dalian University of Technology, China, in 2012 and 2018, respectively. Currently, he is a Professor with the College of Intelligence and Computing, Tianjin University, China. He was a Research Assistant with NUDT, China; a Visiting Student in Electrical and Computer Engineering (ECE) with the University of Toronto, Canada; a Postdoctoral Researcher with HKUST, Hong Kong; and a Researcher with Huawei Theory Lab, Hong Kong. His research interests include datacenter networks and cloud computing. His research papers were published in many prestigious journals and conferences, including IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE International Conference on Computer Communications (INFOCOM); IEEE International Conference on Distributed Computing Systems (ICDCS); International Conference on Parallel Processing (ICPP). He was a recipient of the CCF Outstanding Doctoral Dissertation Award 2020.

work and cloud computing. His research papers were published in many prestigious journals and conferences, including IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE International Conference on Computer Communications (INFOCOM); IEEE International Conference on Distributed Computing Systems (ICDCS); International Conference on Parallel Processing (ICPP). He was a recipient of the CCF Outstanding Doctoral Dissertation Award 2020.



Keqiu Li (Fellow, IEEE) received the bachelor's and master's degrees from the Department of Applied Mathematics, Dalian University of Technology, in 1994 and 1997, respectively, and the Ph.D. degree from the Graduate School of Information Science, Japan Advanced Institute of Science and Technology, in 2005. Currently, he is a Full Professor and the Dean of the College of Intelligence and Computing, Tianjin University, China. He has more than 150 papers published in prestigious journals or conferences such as IEEE/ACM TRANSACTIONS

ON NETWORKING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE International Conference on Computer Communications (INFOCOM); IEEE International Conference on Network Protocols (ICNP). He was a recipient of the National Science Foundation for Distinguished Young Scholars of China. He keeps working on the topics of blockchain, mobile computing, datacenter, and cloud computing.